

Audit Backend

Bugs non corrigeables depuis le frontend - 14 juillet 2026

Ce document liste les bugs identifiés dans le backend Django qui provoquent des erreurs 500 ou des comportements inattendus. Ces bugs ne peuvent pas être corrigés côté frontend - ils nécessitent une modification du code Python.

Bug #1 - Race condition sur l'occupation des tables

MOYEN

Fichier `apps/commandes/models.py`

Lignes 248 et 275 (methode `save()` de `Commande`)

Le `save()` de `Commande` lit l'ancien état avec `Commande.objects.get(pk=self.pk)` sans verrou. Si deux requêtes arrivent en même temps sur la même commande (ex: deux serveurs changent la table), l'état `is_occupied` peut devenir incohérent : une table reste marquée 'occupée' alors qu'elle est libre, ou inversement.

Correction :

```
# Ligne 248 et 275 - remplacer :
old_instance = Commande.objects.get(pk=self.pk)

# par :
old_instance = Commande.objects.select_for_update().get(pk=self.pk)

# Et envelopper le save() dans une transaction :
from django.db import transaction
with transaction.atomic():
    super().save(*args, **kwargs)
```

Bug #2 - Double archivage = crash `DoesNotExist`

MOYEN

Fichier `apps/commandes/models.py`

Lignes 273-304 (methode `save()` de `Commande`)

Quand une commande passe en `VALIDEE` ou `ANNULEE`, le `save()` crée une copie dans `CommandeHistoric` puis appelle `super().delete()` pour supprimer l'original. Si `save()` est appelé une deuxième fois (signal, middleware, double-click), le second appel fait `Commande.objects.get(pk=self.pk)` sur un objet supprimé -> crash `DoesNotExist`.

Correction :

```
# Ligne 274 - remplacer :
if self.pk:
    old_instance = Commande.objects.get(pk=self.pk)

# par :
if self.pk and Commande.objects.filter(pk=self.pk).exists():
    old_instance = Commande.objects.select_for_update().get(pk=self.pk)
```

Bug #3 - Quantité ligne commande = 1 par défaut (silencieux)

MINEUR

Fichier apps/commandes/serializers.py

Ligne 44

Dans LigneCommandeSerializer.create(), si le champ quantity n'est pas envoye par le client, la ligne est creee avec quantite=1 sans erreur. Le frontend envoie toujours la quantite, donc pas de crash, mais un autre client API pourrait creer des lignes avec une quantite inattendue.

Correction :

```
# Soit ajouter required=True sur le champ serializer :  
quantity = serializers.IntegerField(source='quantite', required=True)  
  
# Soit documenter le default=1 comme comportement voulu.
```

Resume

#	Probleme	Fichier	Severite	Status
1	Race condition tables	models.py:248,275	Moyen	A faire
2	Double archivage crash	models.py:273-304	Moyen	A faire
3	Quantite default 1	serializers.py:44	Mineur	A faire